

Testare online (Quiz)

Mirth Kevin - Calculatoare

Contents

1	Introducere	2
1.1	Context General	2
1.2	Obiective	2
1.3	Lista Moscow	2
1.4	Cazuri de utilizare	2
2	Arhitectura	3
2.1	Front-End - React	3
2.2	Back-End - Spring Boot	3
2.3	Comunicare intre Front-End si Back-End	3
2.4	Baza de Date - MySQL	3
2.5	Beneficii ale acestei arhitecturi	4
3	Implementare Frontend	4
3.1	Tehnologii Frontend utilizate	4
3.2	Structura componentelor	4
3.2.1	Componente folosite	4
3.3	Utilizarea aplicatiei din perspectiva Utilizatorului	5
3.4	Utilizarea aplicatiei din perspectiva Administratorului	7
4	Implementare Back-End	9
4.1	Tehnologii backend	9
4.1.1	Spring Boot	9
4.1.2	Spring Data JPA	9
4.1.3	Dependency Injection (DI)	9
4.1.4	Spring Security	10
4.1.5	MySql	10
4.1.6	Maven	10
4.1.7	REST API	10
4.2	Gestionarea cererilor HTTP	11
5	Concluzii	11

1 Introducere

1.1 Context General

Aplicația mea își propune să ofere o soluție practică pentru realizarea testelor de tip grilă, indiferent de domeniul de studiu. Ideea principală este de a ușura procesul de creare și susținere a testelor, făcând totul disponibil online, într-o interfață simplă și ușor de folosit.

1.2 Obiective

Principalul obiectiv al aplicației este de a oferi o modalitate modernă de evaluare, atât pentru profesori sau administratori, cât și pentru studenți sau elevi. Prin realizarea acestei aplicații se urmărește:

- crearea și administrarea testelor online într-un mod cât mai simplu;
- posibilitatea ca utilizatorii să adauge, editeze sau ștergă întrebări;
- efectuarea testelor direct din platformă;
- afișarea rezultatelor și, eventual, trimiterea unor notificări la finalizarea unui test.

1.3 Lista Moscow

Pentru a stabili ce funcționalități sunt esențiale și ce pot fi adăugate pe parcurs, am folosit metoda Moscow, care împarte cerințele în patru categorii:

- **Must have** – funcții care trebuie neapărat să existe;
- **Should have** – funcții importante, dar nu esențiale;
- **Could have** – funcții optionale, care pot fi adăugate ulterior;
- **Won't have** – funcții care nu vor fi incluse momentan.

Must have	Should have	Could have	Won't have
Crearea și completarea testelor	Vizualizarea rezultatelor	Posibilitatea de a edita testele deja create	Rapoarte detaliate pentru fiecare utilizator
Adaugarea și ștergerea întrebărilor	Notificarea la finalizarea unui test	Filtrarea testelor după domeniu	Întrebări diferite pentru fiecare utilizator
Autentificarea utilizatorilor			

1.4 Cazuri de utilizare

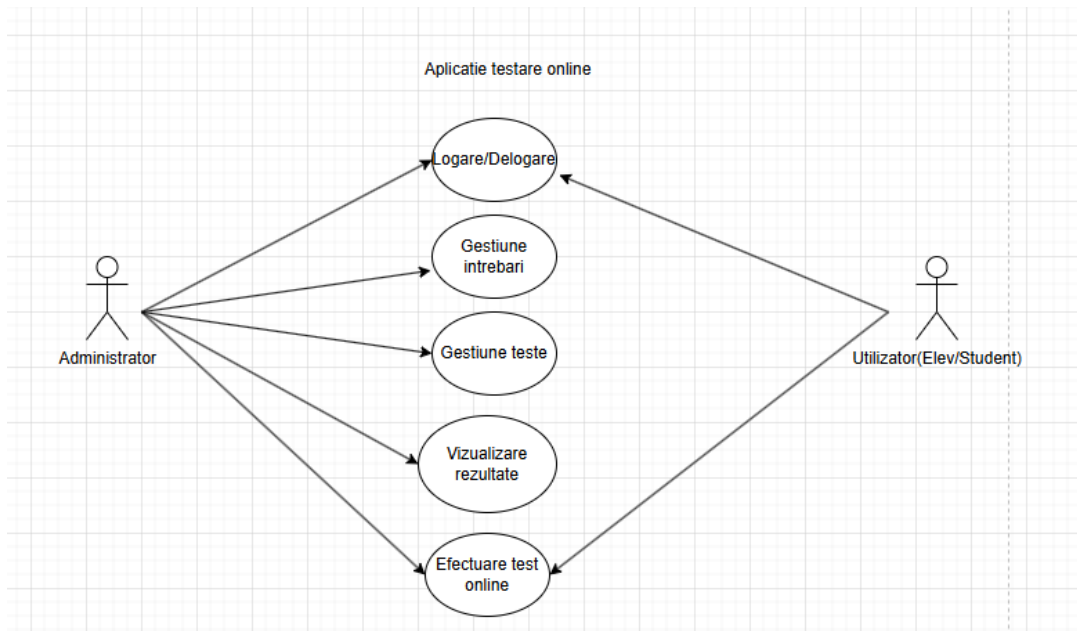
Aplicația este gândită pentru două tipuri principale de utilizatori: administratorul, care creează și gestionează testele, și utilizatorul obișnuit (student/elev), care poate susține testele online. Fiecare dintre ei are un set diferit de acțiuni pe care le poate realiza în aplicație.

Administratorul:

- se poate autentifica în sistem
- adaugă, editează sau șterge întrebări și teste
- poate vedea rezultatele și notificările.

Utilizatorul(Elev/Student)

- se poate loga și efectua testele disponibile
- poate vizualiza rezultatul obținut după finalizarea testului



2 Arhitectura

Arhitectura aplicației este gândită astfel încât să ofere performanță bună, stabilitate și o experiență plăcută pentru utilizator. Aplicația este construită pe o structură client-server, unde partea de front-end este realizată în React, iar partea de back-end folosește Spring Boot. Împreună, aceste tehnologii moderne oferă o aplicație rapidă, ușor de extins și ușor de întreținut.

2.1 Front-End - React

Pentru partea de interfață, am ales React deoarece este una dintre cele mai populare biblioteci pentru dezvoltarea aplicațiilor web moderne. React oferă o organizare pe componente, ceea ce face codul mai curat și mai ușor de modificat. În plus, actualizările din pagină se fac foarte rapid, fără reîncărcarea completă a site-ului. Astfel, utilizatorii pot naviga și completa testele într-un mod fluent și intuitiv.

2.2 Back-End - Spring Boot

Partea de server este construită cu ajutorul Spring Boot, care oferă un cadru stabil și ușor de folosit pentru dezvoltarea aplicațiilor Java. Spring Boot simplifică foarte mult procesul de configurare și permite crearea rapidă a serviciilor REST, folosite pentru a trimite și primi date între server și client. De asemenea, prin Spring Data JPA, se asigură o interacțiune simplă și clară cu baza de date MySQL, fără a fi nevoie să scriem manual multe interogări SQL.

2.3 Comunicare între Front-End și Back-End

Comunicarea dintre React și Spring Boot se face cu ajutorul bibliotecii Axios, care permite trimiterea cererilor HTTP (GET, POST, PUT, DELETE) către server. Axios gestionează răspunsurile într-un mod asincron, ceea ce ajută la încărcarea mai rapidă a datelor și la o experiență mai plăcută pentru utilizator. Această metodă de comunicare este simplă, sigură și foarte eficientă pentru aplicațiile web moderne.

2.4 Baza de Date - MySQL

Pentru stocarea datelor, aplicația folosește MySQL, un sistem de baze de date relațional foarte cunoscut și ușor de utilizat. MySQL este o alegere potrivită pentru proiecte de dimensiuni medii, fiind stabil, rapid și compatibil cu Spring Boot. Prin Spring JPA, operațiile de tip CRUD (Create, Read, Update, Delete) se realizează simplu, fără a scrie interogări complicate, iar conexiunea cu baza de date este gestionată automat de framework.

2.5 Beneficii ale acestei arhitecturi

- **Scalabilitate:** Aplicația poate fi extinsă ușor, adăugând noi module sau funcționalități.
- **Performanta:** Comunicarea eficientă dintre front-end și back-end, împreună cu utilizarea MySQL, asigură timpi de răspuns buni.
- **Usurinta in dezvoltare:** React și Spring Boot sunt framework-uri populare, cu multe resurse și suport din comunitate.
- **Flexibilitate:** Arhitectura modulară permite modificarea și extinderea aplicației fără a afecta celelalte părți.

3 Implementare Frontend

Dezvoltarea interfeței utilizatorului pentru aceasta aplicație de quiz a necesitat o implementare riguroasă. Partea de front-end constituie punctul principal de contact între utilizator și aplicație, iar pentru a asigura o navigare fluentă și o interacțiune plăcută, am acordat atenție sporită selecției de tehnologii și organizarea structurală a codului.

3.1 Tehnologii Frontend utilizate

Pentru interfața aplicației am ales React, deoarece este unul dintre cele mai populare framework-uri pentru dezvoltare web. React mi-a permis să organizez codul în componente reutilizabile, ceea ce face aplicația mult mai ușor de înțeles și de modificat ulterior.

Aplicația folosește tehnologiile de bază ale web-ului: HTML pentru structura paginilor, JavaScript pentru funcționalități și CSS pentru design. Am creat stiluri personalizate pentru fiecare componentă importantă, astfel încât interfața să arate uniform și profesional.

Pentru comunicarea cu serverul am folosit Axios, o bibliotecă JavaScript care face mai simplu procesul de trimitere și primire a datelor. Axios gestionează automat cererile HTTP și lucrează bine cu formatul JSON, ceea ce îmi permite să primesc date de la backend într-un mod eficient.

3.2 Structura componentelor

Structura componentelor este organizată modular pentru a facilita întreținerea și extinderea ulterioară. Componentele create sunt concepute să fie reutilizabile, favorizând coerența și eficiența în dezvoltarea front-end-ului.

3.2.1 Componente folosite

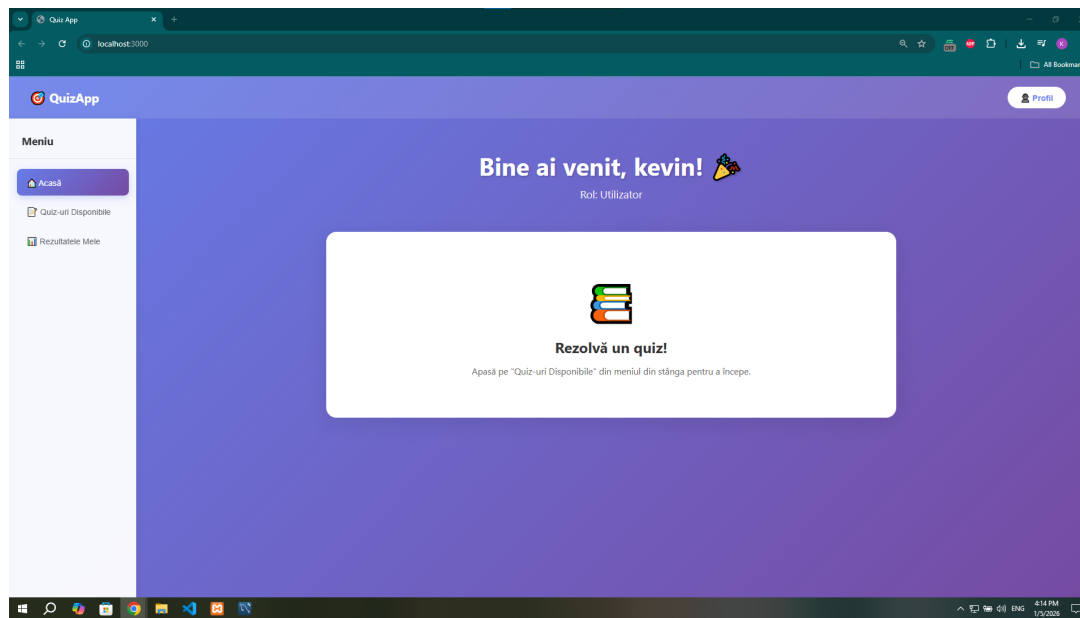
Principalele componente folosite în cadrul frontend-ului sunt prezente în tabelul următor.

Numele componentei	Rolul componentei	Utilizată la endpoint-ul
App	Componenta centrală a aplicației React care gestionează rutarea	Toate
Login	Pagină de autentificare pentru conectarea utilizatorilor	/login
Register	Pagină pentru crearea unui cont nou de utilizator	/register
Dashboard	Pagina principală după autentificare cu opțiuni disponibile	/
CreateQuiz	Formular pentru crearea unui quiz nou cu întrebări	/create-quiz
MyQuizzes	Afișează toate quiz-urile create de utilizator	/my-quizzes
AvailableQuizzes	Listează quiz-urile disponibile pentru completare	/available-quizzes
TakeQuiz	Interfață pentru completarea unui quiz	/take-quiz/:id

Numele componentei	Rolul componentei	Utilizată la endpoint-ul
QuizHistory	Afișează istoricul quiz-urilor completate cu scoruri	/history
Profile	Pagină cu informațiile profilului utilizatorului	/profile

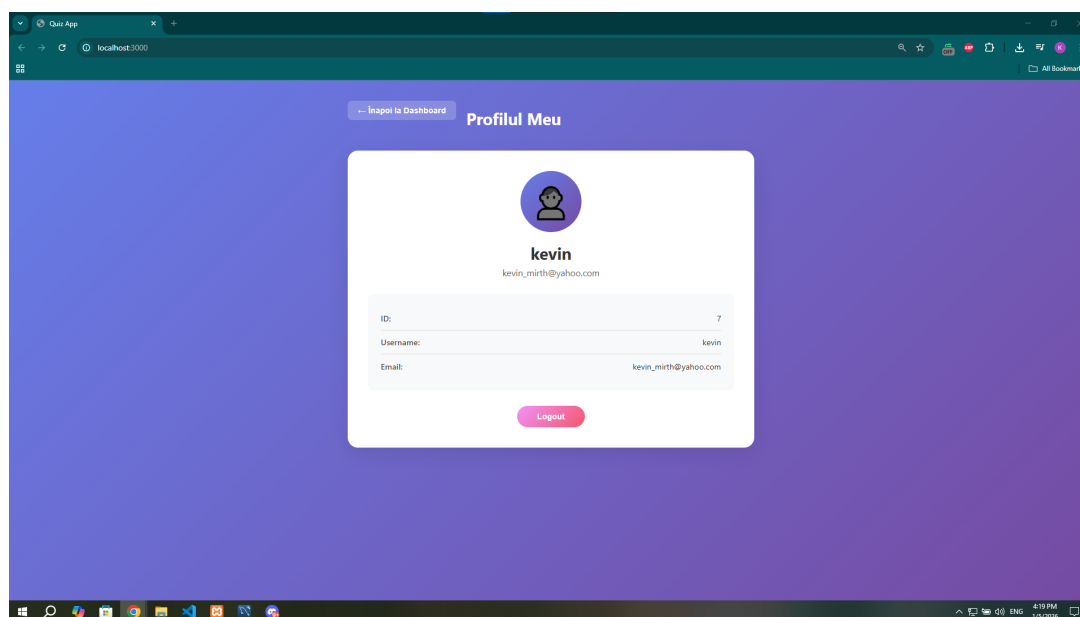
3.3 Utilizarea aplicatiei din perspectiva Utilizatorului

1. Home Page



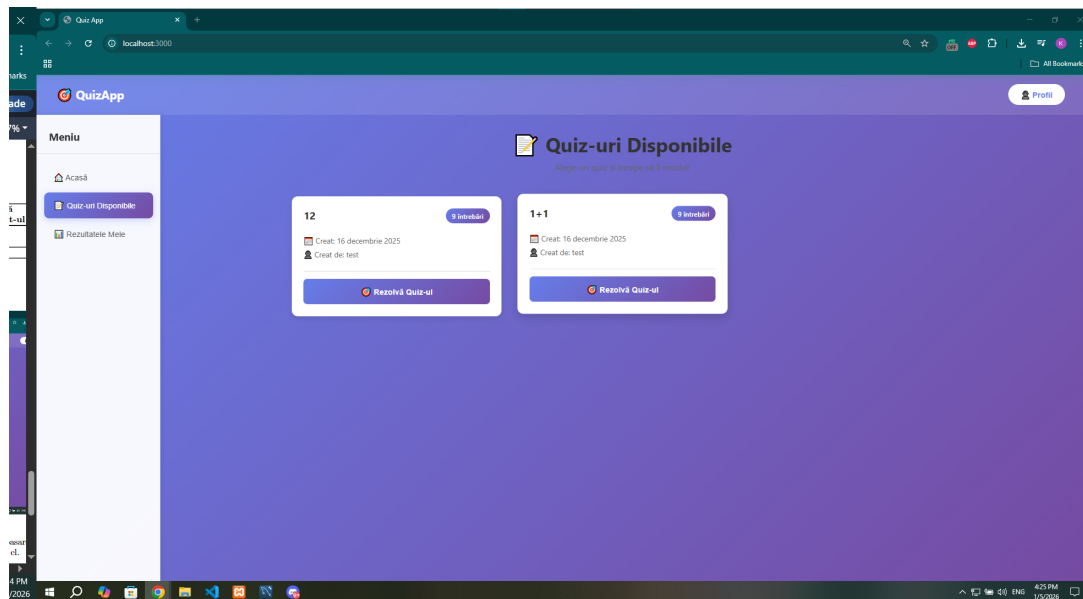
2. Profil

De pe homepage se poate observa in coltul din dreapta sus butonul "Profil", la apasarea acestuia utilizatorul va fi redirectionat pe pagina profil, unde acesta poate sa vada detalii despre el.



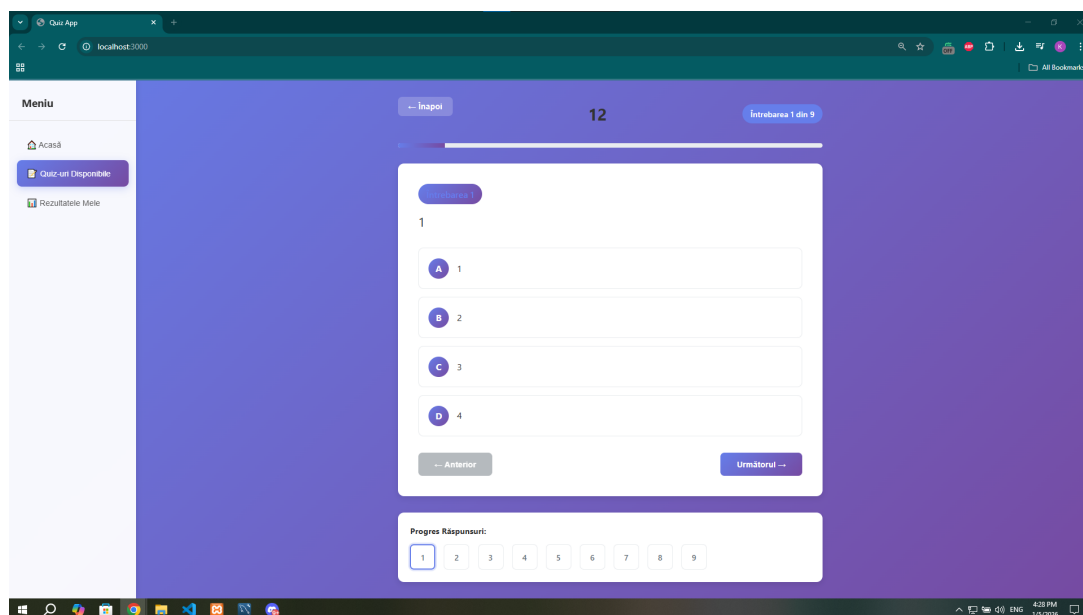
3. Quiz-uri Disponibile

Pe sidebar-ul de pe homepage se găsește butonul "Quiz-uri disponibile" iar la apăsarea acestuia utilizatorul este redirectionat pe pagina cu toate quizurile create de către administratori.



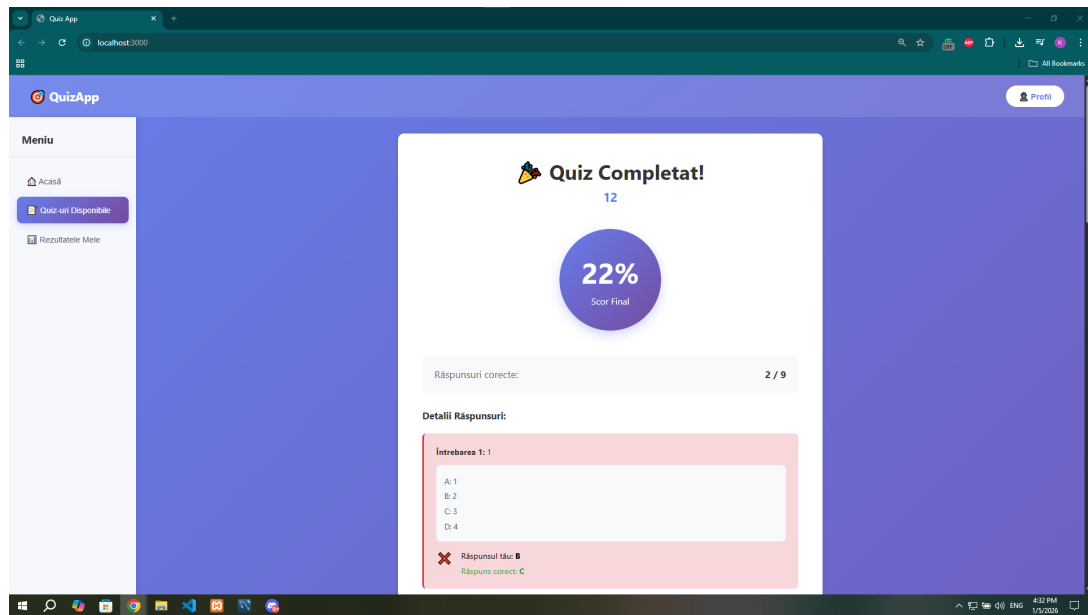
4. Rezolva quiz

Pentru a rezolva unul dintre quiz-urile disponibile utilizatorul trebuie să apăsă butonul "Rezolva quiz" din dreptul quiz-ului dorit. Odată apăsă butonul, acesta va fi redirectionat direct la quiz și poate să-l rezolve.



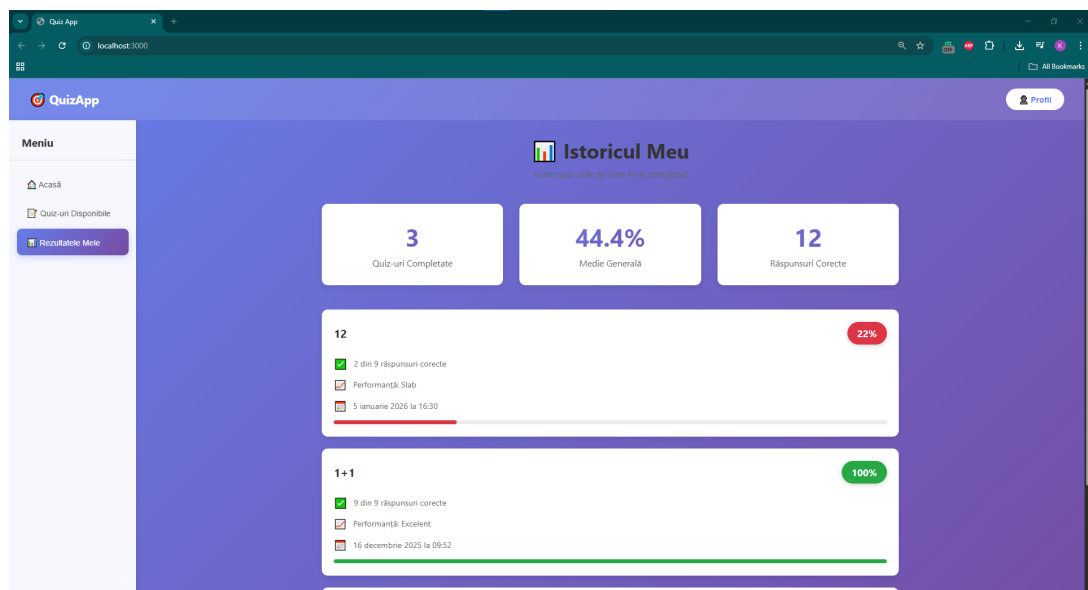
5. Rezultatul

La finalizarea unui quiz, utilizatorul va fi redirectionat la o nouă pagină unde aceasta va vedea rezultatul, iar tot odată numărul de răspunsuri corecte, respectiv gresite, dar și fiecare întrebare în parte cu răspunsul său sau cel corect.



6. Rezultatele mele

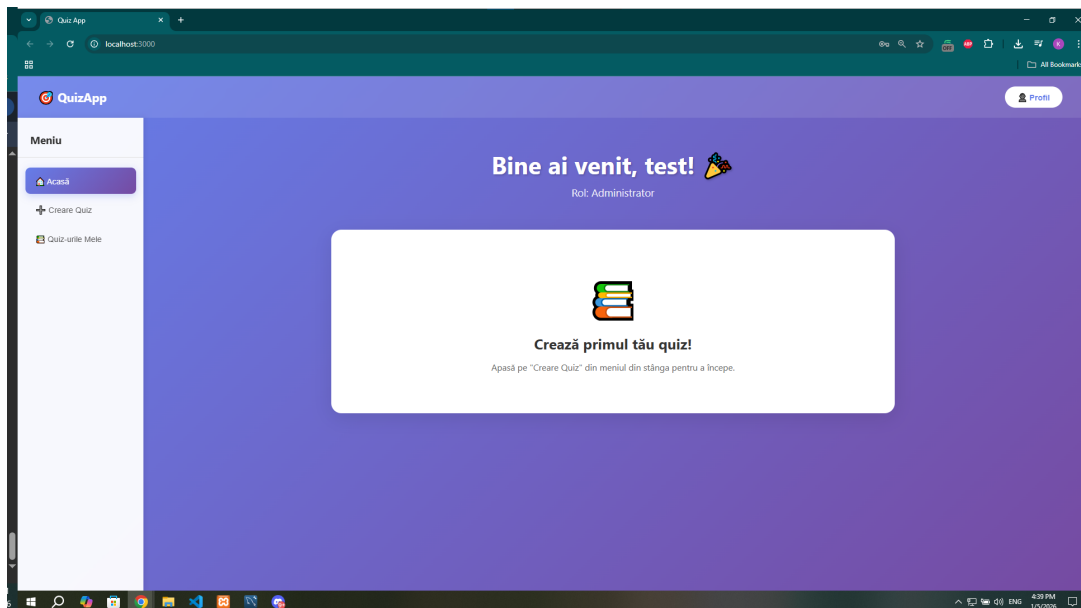
Tot pe sidebar se afla si butonul "Rezultaatele Mele", unde utilizatorul poate sa vada o mica statistica a quizurilor sale. Un numar de quiz-uri facute, media rezultatelor, dar si numarul total de raspunsuri corect, cat si istoricul quizurilor cu mici detalii despre acestea.



3.4 Utilizarea aplicatiei din perspectiva Administratorului

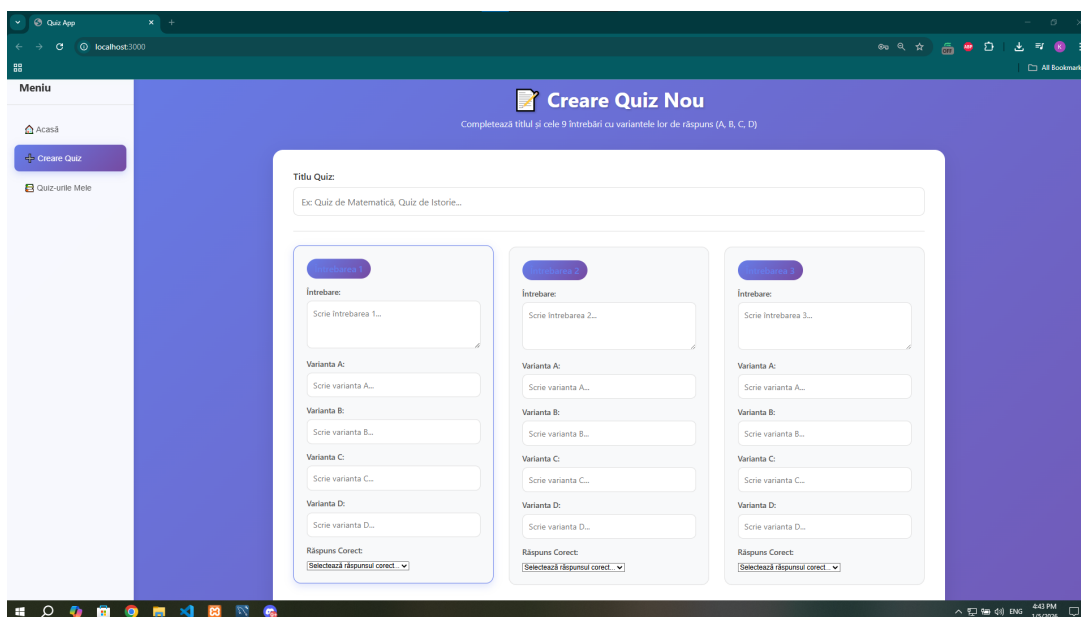
1. Homepage

Pe homepage-ul administratorul se pot vedea toate quiz urile create de acesta.



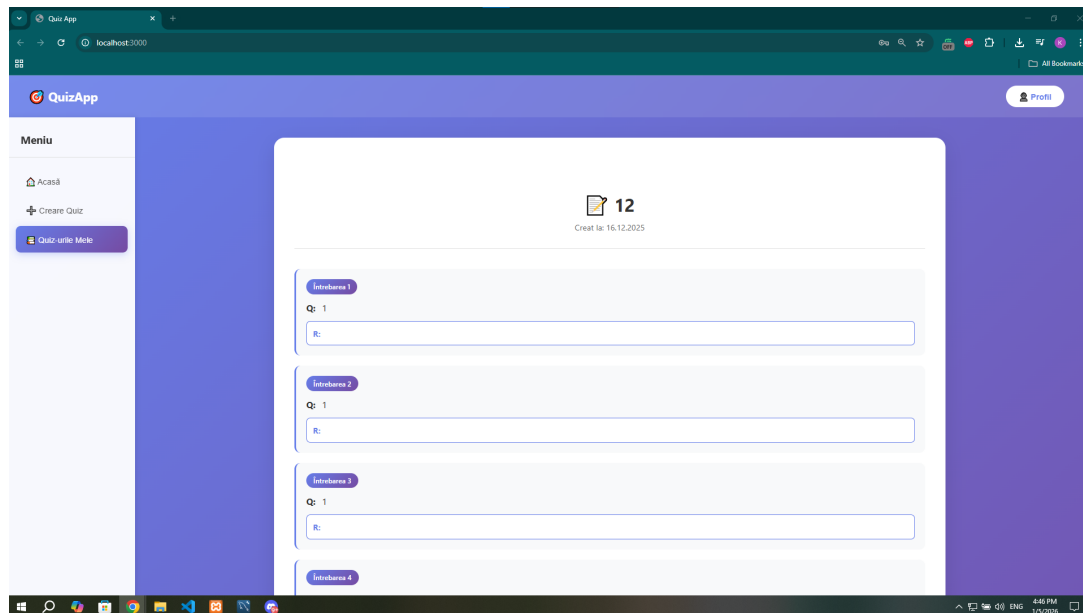
2. Creare quiz

La apăsarea butonului "Creare quiz" de pe sidebar administratorul este redirectionat pe pagina de creare de quizuri, unde acesta primește indicații foarte clare despre cum creează un quiz.



3. Quiz-urile mele

Pe pagina "quizurile mele", administratorul vede toate quiz-urile create de acesta, unde acesta poate să vadă detaliile quiz-ului.



4 Implementare Back-End

4.1 Tehnologii backend

Back-end ul aplicatiei ofera suportul necesar pentru functionalitatile aplicatiei, gestioneaza interactiunile, prelucreaza datele. Am ales tehnologiile pentru backend, in asa fel incat sa fie cat mai eficient, scalabil si usor in dezvoltare.

4.1.1 Spring Boot

Spring Boot este framework-ul principal ales pentru dezvoltarea backend-ului aplicatiei de quiz. Alegerea acestuia se datoreaza capacitatii sale de a simplifica configurarea si dezvoltarea aplicatiilor Java, oferind o abordare convention-over-configuration care reduce semnificativ timpul de dezvoltare si mentine un nivel inalt de consistenta in proiect.

4.1.2 Spring Data JPA

Java persistence API reprezinta standardul Java pentru gestionarea persistentei datelor in aplicatii. Integrarea Spring Data JPA in proiect aduce beneficii semnificative in ceea ce priveste manipularea datelor in baza de date, permitand lucrul cu obiecte Java si maparea acestora in tabele relationale.

Avantaje Spring Data JPA

- Abordare Obiect-Relatională: JPA permite lucrul cu obiecte Java in loc de interogari SQL directe, oferind o abordare mai orientate pe obiect in gestionarea datelor.
- Reducerea codului boilerplate: Repo urile JPA genereaza automat implementari pentru operatiunile CRUD, eliminand necesitatea scrierii de cod repetitiv.
- Reutilizabilitatea si Consistenta: Utilizarea entitatilor JPA si a repository-urilor abstractizeaza detaliile tehnice ale manipulării datelor, facilitand reutilizarea si mentinerea consistentei.

4.1.3 Dependency Injection (DI)

Dependency Injection este un concept fundamental in Spring Framework, unde obiectele nu isi creeaza direct dependentele, ci le primesc injectate din exterior. Spring faciliteaza implementarea acestui pattern prin anotari simple si intuitive.

Avantaje

- Decuplare si testare usoara: Dependency Injection promoveaza decuplarea intre componente, facilitand testarea unitara prin Injectarea de mock objects.

- Gestionarea automata a dependentelor: Spring gestioneaza automat ciclul de viata al componentelor, asigurand crearea si distrugerea corespunzatoare.
- Configurarea declarativa: Dependetele sunt configurate prin anotari, eliminand necesitatea configurarilor XML complexe.

4.1.4 Spring Security

Spring Security este framework-ul utilizat pentru gestionarea autentificarii si autorizarii in aplicatie. Implementarea se bazeaza pe JSON Web Tokens (JWT), care ofera o metoda sigura si eficienta de autentificare stateless.

JSON Web Token (JWT) este un standard deschis (RFC 7519) care defineste un format compact de transmitere a informatiilor intre parti sub forma de obiecte JSON. In contextul aplicatiei de quiz, JWT este utilizat pentru gestionarea sesiunilor utilizatorilor.

Generarea token-ului JWT:

- Dupa autentificarea cu succes, serverul genereaza un token JWT care contine informatii despre utilizator (id, username, rol).
- Token-ul este semnat digital folosind o cheie secreta pentru a garanta integritatea si autenticitatea datelor.

Avantaje ale JWT:

- Stateless si Eficienta: JWT permite aplicatiei sa fie stateless, eliminand necesitatea de a mentine sesiuni la nivel de server. Token-ul contine toate informatiile necesare pentru autentificare.
- Scalabilitate: Abordarea stateless faciliteaza scalarea orizontala a aplicatiei, fara a fi necesara sincronizarea sesiunilor intre servere.
- Securitate: Token-ul este semnat digital, garantand integritatea si autenticitatea informatiilor. Utilizarea BCrypt pentru criptarea parolelor asigura protectia datelor utilizatorilor.
- Flexibilitate: JWT poate include informatii personalizate in payload, reducand numarul de interogari catre baza de date.

Prin implementarea JWT in aplicatia de quiz, se obtine o metoda eficienta si sigura de gestionare a autentificarii si autorizarii, oferind un mecanism robust pentru securitatea aplicatiei.

4.1.5 MySql

Am optat pentru MySQL ca sistem de gestionare a bazelor de date datorita stabilitatii, performantei si popularitatii sale in ecosistemul Java. MySQL ofera suport excelent pentru tranzactii ACID, indexare eficienta si gestionarea optima a relatiilor intre tabele.

MySQL integreaza perfect cu Spring Data JPA, permitand maparea automata a entitatilor Java in tabele si oferind performante excelente pentru operatiunile CRUD.

4.1.6 Maven

Maven este utilizat pentru gestionarea dependentelor si construirea proiectului. Prin intermediul Maven, gestionam eficient toate dependentele externe ale aplicatiei, inclusiv Spring Boot, Spring Security, MySQL Connector si bibliotecile JWT.

Maven faciliteaza procesul de build, testare si deployment al aplicatiei, asigurand consistenta dependentelor in toate mediile de dezvoltare.

4.1.7 REST API

Backend-ul aplicatiei expune un REST API complet care permite comunicarea cu frontend-ul. API-ul este structurat logic pe controllere care gestioneaza diferite functionalitati.

Toate endpoint-urile returneaza raspunsuri in format JSON si utilizeaza coduri HTTP standard pentru indicarea succesului sau esecului operatiunilor.

Aceste tehnologii formeaza baza solida a backend-ului aplicatiei de quiz, contribuind la dezvoltarea si intretinerea eficienta a unei solutii robuste si scalabile. Prin alegerea acestor instrumente si tehnologii moderne, aplicatia beneficiaza de securitate avansata, performanta optima si usurinta in dezvoltare si mentenanta.

4.2 Gestionarea cererilor HTTP

In cadrul aplicatiei, gestionarea cererilor HTTP reprezinta un aspect fundamental al backend-ului, facilitand comunicarea eficienta intre frontend si server. Ne bazam pe principiile arhitecturii RESTful pentru a proiecta API-urile noastre, asigurand o structura clara si usor de inteles.

Am implementat o serie de endpoint-uri care ofera functionalitati diverse in functie de cerintele aplicatiei noastre. Acestea sunt expuse printr-un set de endpoint-uri bine definite, care accepta metode HTTP standard, precum GET, POST, PUT si DELETE. Toate endpoint-urile sunt centralizate in tabelul urmator:

Endpoint		Metoda HTTP	Utilizare
2* http://localhost:8080/api/auth			
	/login	POST	Autentificare in aplicatie
	/register	POST	Creaza un cont nou
8* http://localhost:8080/api/quiz			
	/available	GET	Furnizeaza toate quiz-urile disponibile
	/my-quizzes	GET	Furnizeaza quiz-urile create de utilizator
	/create	POST	Creaza un quiz nou
	/ {id}	GET	Furnizeaza detaliile unui quiz specific
	/ {id}/questions	GET	Furnizeaza intrebarile asociate unui quiz
	/submit	POST	Trimite raspunsurile si calculeaza scorul
	/results	GET	Furnizeaza istoricul rezultatelor utilizatorului
	/delete/ {id}	DELETE	Sterge un quiz existent

Table 2: Endpoint-urile API-ului aplicatiei de quiz

Link Github: <https://github.com/KevinMirth/quiz.git>

5 Concluzii

Aplicatia dezvoltata reprezinta o platforma completa si functionala pentru gestionarea quiz-urilor online, oferind o baza solida pentru viitoare imbunatatiri si extinderi ale functionalitatilor.

Dezvoltarea ulterioara a aplicatiei poate include diverse directii de evolutie, incepand cu aspecte simple precum: adaugarea posibilitatii de editare a quiz-urilor existente, implementarea unui sistem de taguri pentru o mai buna organizare a continutului, integrarea de optiuni de cautare avansata si filtrare dupa diverse criterii, sau dezvoltarea unui modul de export al rezultatelor in formate populare.

Pe planul functionalitatilor avansate, aplicatia ar putea fi imbunatatita prin: implementarea unui sistem de randomizare a intrebarilor si raspunsurilor pentru fiecare sesiune de quiz, adaugarea unei sectiuni de analiza si raportare detaliata pentru evaluarea performantelor, crearea unui sistem de badge-uri si trofee pentru motivarea utilizatorilor, sau dezvoltarea unui panel administrativ avansat cu statistici in timp real si grafice interactive.

Extinderi semnificative ar putea include integrarea cu servicii externe de autentificare (Google, Facebook), implementarea unui sistem de notificari push pentru diverse evenimente, dezvoltarea unei versiuni mobile native pentru iOS si Android, sau chiar integrarea de elemente de inteligenta artificiala pentru generarea automata de intrebari sau recomandari personalizate.

Din perspectiva tehnica, aplicatia ar putea beneficia de optimizari precum: implementarea de mecanisme de caching distribuit pentru reducerea timpilor de raspuns, utilizarea de message queues pentru procesarea asincron a task-urilor intensive, adaugarea unui sistem complet de monitorizare si alertare, sau implementarea de strategii de backup automat si disaster recovery.

In final, aplicatia dezvoltata demonstreaza o integrare eficienta a tehnologiilor moderne web, oferind o experienta fluida si sigura pentru toti utilizatorii. Frontend-ul construit cu React asigura o interfata dinamica si prietenoasa, cu o navigare intuitiva si un design responsive care se adapteaza perfect la orice dimensiune de ecran.

Backend-ul dezvoltat pe platforma Spring Boot ofera o arhitectura robusta si usor de mentinut, beneficiind de ecosistemul vast al framework-ului Spring. Sistemul de securitate implementat prin Spring Security si autentificare JWT garanteaza protectia datelor utilizatorilor si controlul strict al accesului la resurse, eliminand vulnerabilitatile comune de securitate.

Utilizarea MySQL ca sistem de management al bazei de date asigura persistenta fiabila a informatiilor, in timp ce Spring Data JPA reduce semnificativ complexitatea codului prin generarea automata a query-urilor si gestionarea eficienta a entitatilor. Structurarea API-ului conform principiilor REST asigura o comunicare clara si predictibila intre componentele aplicatiei, facilitand integrarea viitoare cu alte sisteme.

Flexibilitatea arhitecturii alese permite extinderea rapida a aplicatiei prin adaugarea de noi module si functionalitati, fara a necesita modificari majore ale codului existent. Separarea clara a responsabilitatilor intre layere (presentation, business logic, data access) asigura mentenabilitatea pe termen lung a proiectului.

In concluzie, aplicatia realizata nu doar ca satisface cerintele initiale de a oferi o platforma functionala pentru crearea si completarea de quiz-uri online, dar pune si bazele unei solutii scalabile si moderne, pregatita pentru evolutii viitoare. Tehnologiile selectate si principiile de design aplicate garanteaza ca aplicatia poate fi continuu imbunatatita si adaptata la cerintele in continua schimbare ale utilizatorilor si ale industriei software.